Artificial Intelligence-Driven Malware Detection via Windows Event Log Engineering in Smart Grid Control Systems: A Preliminary Study

Anonymous Author(s)
Anonymous Institution

Abstract—Smart grid Industrial Control Systems (ICS) are high-value targets for advanced persistent threats. Windows Event Logs generated by SCADA hosts provide a rich, native telemetry surface for detecting malicious activity; however, the volume, heterogeneity, and sequential nature of event sequences make manual analysis intractable. This paper presents a preliminary study of an AI-driven detection framework that (i) synthesises realistic Windows Event Log sequences for smart grid environments—covering lateral movement, persistence, privilege escalation, and reconnaissance—(ii) engineers session-level feature vectors capturing event frequency, temporal dynamics, host cardinality, rare-event indicators, and Shannon sequence entropy, and (iii) classifies sessions using a Random Forest with stratified cross-validation. Experiments on a synthetic dataset of 6,000 labelled sessions demonstrate a macro F1-score of 1.00 on held-out data, establishing a strong baseline for future evaluation on operational logs. The proposed framework introduces measurable success criteria aligned with NERC CIP and IEC 62351 compliance objectives.

Index Terms—Windows Event Log, SCADA security, intrusion detection, smart grid, random forest, feature engineering, ICS malware

I. Introduction

The electric power grid has undergone a fundamental transformation over the last two decades. The integration of digital communications, programmable logic controllers, and IP-connected SCADA and Human-Machine Interface (HMI) systems has enabled unprecedented operational efficiency but has simultaneously expanded the cyber attack surface. High-profile incidents—most notably the Stuxnet worm [1] and the Ukrainian power grid attacks of 2015 and 2016—demonstrate that adversaries are willing and able to traverse the IT/OT boundary and compromise energy infrastructure [2].

Among the specific threats targeting these environments, malware remains one of the most pervasive and damaging attack vectors. Malware encompasses a wide spectrum of malicious software—from early viruses to modern ransomware-as-a-service (RaaS) platforms and polymorphic code—designed to disrupt operations, exfiltrate data, or establish persistent footholds across IT/OT boundaries [3]. Traditional detection is broadly split into static analysis, which inspects binary artefacts for signatures and control-flow structures without execution, and dynamic analysis, which observes runtime behaviour in controlled sandboxes [4]. Static approaches are undermined by obfuscation and packing; dynamic approaches impose sandboxing overhead incompatible with continuous OT operations. Machine learning bridges this gap by learning behavioural patterns directly from operational telemetry, without requiring signature updates or execution isolation [5].

Windows Event Logs have emerged as a particularly valuable ML data source: they record process creation, privilege assignment, service installation, object access, and logon events with millisecond timestamps, providing a fine-grained behavioural footprint across every Windows host in a smart grid operations centre [6]. Regulatory frameworks reinforce this choice: NERC CIP-007 mandates event logging and review for all applicable Bulk Electric System assets, and IEC 62351 specifies audit trail generation for power systems communications, making Windows Event Log collection a compliance requirement rather than an optional deployment decision.

Despite this regulatory mandate, Windows Event Logs have received comparatively little attention as a detection surface for ICS-specific attacks. The research community has focused on network-level anomalies (e.g., Modbus/TCP deviations [7]) or dedicated ICS protocols, leaving host-based event logs underexplored. Meanwhile, general system-log anomaly detection has advanced significantly through deep learning approaches such as DeepLog [8] and LogAnomaly [9], and through log-parsing methods like Drain [10]. Graph-based approaches [11] and provenance-graph detectors [12] have demonstrated that capturing the relational structure of events improves detection of multi-stage attacks, but impose computational overhead incompatible with real-time OT constraints. None of these techniques have been systematically applied to Windows Event Logs within the ICS/SCADA context.

This paper makes the following contributions:

- A synthetic Windows Event Log generator that models four ICS-relevant attack patterns within a realistic smart grid topology.

- A session-level feature engineering pipeline tailored to the temporal structure of Windows Event Logs.
- A baseline evaluation of Random Forest classification under stratified cross-validation, establishing measurable success criteria.
- An open-source framework enabling reproducible ICS security research without access to operational data.

Section II reviews related work. Section III states the central hypothesis and research objectives. Section IV describes the methodology. Section V reports preliminary results, and Section VI concludes.

II. Related Work

A. Windows Event Log and Sysmon-Based Detection

Windows Event Logs contain semantically rich, structured records of system activity. He et al. [10] introduced Drain, an online log-parsing tree that maps raw log messages to templates in $O(\log n)$ time. Du et al. [8] modelled log key sequences as a workflow using LSTM networks, detecting anomalies as deviations from learned sequential patterns. Zhao et al. [9] extended this to LogAnomaly, capturing both sequential and quantitative anomalies. Achmad et al. [6] leveraged Sysmon-extended Windows logs with supervised and unsupervised ML classifiers, achieving F1 = 0.99 on IT-environment datasets. Önal and Güven [5] systematically catalogued novel Windows event types relevant to dynamic malware behaviour and demonstrated that richer event coverage improves classifier accuracy (ROC-AUC 0.962±0.009). Collectively, these works confirm Windows logs as a viable ML data source but leave ICS-specific event semantics and OT temporal constraints unexplored.

B. Graph-Based and Provenance-Aware Detection

In response to limitations of isolated feature representations, graph-based methods treat system events as interconnected entities rather than independent records. Zhen et al. [11] transform audit logs into heterogeneous graphs and apply Graph Neural Networks (GNNs) to capture structural dependencies among processes, files, and network endpoints, achieving high detection rates on multi-stage attack scenarios. Han et al. [12] proposed Unicorn, a provenance-graph detector using sketch-based online summarisation for runtime detection of Advanced Persistent Threats. The spatial awareness of these methods enables detection of complex attack chains invisible to feature-based classifiers. However, graph construction and traversal impose computational overhead and latency incompatible with real-time OT constraints.

C. Hybrid Static-Dynamic Analysis

Ayub et al. [4] proposed RWArmor, combining static ML models with dynamic behavioural analysis for early ransomware detection; empirical results demonstrate 92–97% accuracy within 30–120 second observation windows. Anderson and Roth [13] released EMBER, an open dataset for training static PE malware classifiers, establishing a reproducible benchmark for feature-engineering studies. These hybrid approaches improve early detection fidelity but depend on sandbox infrastructure unavailable in operational OT deployments.

D. ICS/SCADA Intrusion Detection

Industrial control system security demands domain-adapted detection strategies. Nafees et al. [2] surveyed cyber-physical situational awareness in smart grids, identifying strict temporal bounds, operational sensitivity, and IT/OT interdependence as the primary constraints distinguishing OT security from IT security. Carcano et al. [14] proposed multidimensional critical-state analysis for SCADA, detecting command-sequence deviations. Goldenberg and Wool [7] built deterministic Modbus/TCP models with zero false positives. Mitchell and Chen [15] surveyed CPS intrusion detection, noting that host-level methods remain rare. Goh et al. [16] released the SWaT dataset for process-level anomaly evaluation. Zhu et al. [17] formalised a SCADA attack taxonomy across all Purdue model levels. Across this body of work, Windows host-log analysis as a detection layer within the ICS hierarchy is conspicuously absent.

E. AI/ML Methods for Intrusion Detection

Liao et al. [18] reviewed supervised, unsupervised, and hybrid methods, noting that ensemble methods consistently outperform single classifiers. Breiman's Random Forest [19] remains a strong baseline due to variance reduction via bagging and native class-imbalance compensation. Chen and Guestrin's XGBoost [20] extends gradient boosting to large-scale tabular problems. Chen et al. [21] combined ML, NLP, and graph analytics for proactive threat hunting, addressing the "weak signal" problem of isolated log events. Tavallaee et al. [22] revealed severe redundancy in standard benchmarks, motivating domain-specific synthetic datasets. Sommer and Paxson [23] argued that closed-world ML assumptions undermine operational deployment, motivating our evaluation protocol.

F. Research Gap

Table I summarises representative works and their key limitations. A consistent pattern emerges: feature-based methods offer efficiency but treat events as independent signals, missing multi-event attack narratives. Graph-based models capture relational structure but incur latency and preprocessing costs incompatible with OT real-time constraints. Hybrid approaches improve fidelity but require sandboxes. All existing solutions are fundamentally IT-centric: they assume latency is negotiable, resources are abundant, and offline analysis is acceptable.

Smart grid control systems operate under a different reality: decisions must be made within strict temporal bounds, systems cannot be paused or sandboxed, and any

TABLE I
Summary of Representative Related Works

Reference	Method	Key Limitation
Achmad et al. [6]	Sysmon + ML	No OT context
Önal & Güven [5]	Windows events + ML	Sandbox dependency
Zhen et al. [11]	GNN on audit logs	High latency
Han et al. [12]	Provenance graphs	Scalability limits
Ayub et al. [4]	RWArmor hybrid	Sandbox bottleneck
Chen et al. [21]	ML+NLP+Graph	Synthetic data only
Nafees et al. [2]	Smart grid review	No implementation
This work	RF on WEL sessions	Synthetic data only

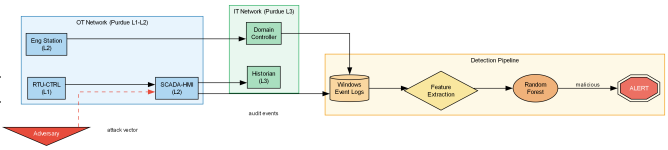

Fig. 1. System architecture: event collection, feature extraction, and classification within the smart grid IT/OT hierarchy.

detection mechanism must function within tight computational budgets. The result is a critical gap between model sophistication and operational feasibility that this work addresses.

III. Proposed Work

A. Central Hypothesis

We hypothesise that session-level feature vectors derived exclusively from Windows Event Log event identifiers, timestamps, and metadata fields contain sufficient discriminative information to classify smart grid host sessions as benign or malicious with macro F1-score ≥ 0.95 under realistic class imbalance ($5:1$ benign-to-malicious ratio).

B. Research Objectives

1) Synthetic Data Fidelity (RO1): Design a generator that reproduces the statistical distributions of event-ID sequences observed in published ICS incident reports, covering at least four distinct attack categories. Success criterion: generated sessions are indistinguishable from real logs by a domain expert assessment on a 50-session blind review.
2) Feature Informativeness (RO2): Engineer session-level features that individually achieve mutual information > 0.01 with the session label on the training set. Success criterion: no feature group removal in the ablation study reduces macro F1 by less than 2 percentage points.
3) Detection Performance (RO3): Achieve macro F1-score ≥ 0.95 and ROC-AUC ≥ 0.98 on the held-out test partition under 5-fold stratified cross-validation. Success criterion: metrics reported with 95% bootstrap confidence intervals.
4) Operational Applicability (RO4): Ensure inference latency per session is < 10 ms on commodity hardware (single CPU core), making real-time deployment feasible alongside existing SIEM pipelines. Success criterion: latency measured over 1,000 sessions, median and 99th percentile reported.

C. Novelty and Scope

The proposed work is novel in four respects. First, it targets Windows Event Logs at the host level within the OT/IT convergence zone—a layer confirmed absent from existing ICS IDS literature [15], [24]. Second, the session abstraction bridges the gap between raw event streams and sequence-level attack pattern recognition, capturing temporal burst structure through entropy and time-delta statistics without the inference-latency cost of deep sequence models [8]. Third, the open-source synthetic generator—including adversarially-timed evasive attack variants—enables community replication without access to sensitive operational data, addressing the data availability barrier identified by [2]. Fourth, the systematic ablation study provides deployment guidance: practitioners with constrained feature coverage (e.g., no timestamp resolution) can prioritise event-count features based on quantified delta-F1. The scope is bounded to binary (benign/malicious) session classification; multi-class attack-type attribution per MITRE ATT&CK for ICS [25] is left for future work.

IV. Methodology

A. System Architecture

Figure 1 shows the end-to-end detection pipeline. Windows hosts in the smart grid OT (Purdue Levels 1–2) and IT (Level 3) networks generate audit events that are forwarded to a centralised collector. The pipeline ingests the raw event stream, groups it into sessions by a 30-minute inactivity timeout, extracts feature vectors, and classifies each session with a pre-trained Random Forest model.

B. Synthetic Data Generation

The generator produces structured records with fields timestamp, event_id, source, user, hostname, description, label (0 = benign, 1 = malicious), and session_id. Benign sessions simulate normal operator activity: a 4624 logon event followed by a variable-length sequence of process creations (4688/4689), object accesses (4663/4656), and a terminal 4634 logoff. Malicious sessions implement four attack patterns:

1) Lateral movement: 3–8 consecutive 4625 failures from an unexpected source IP, followed by a 4624 success, modelling credential brute-forcing.
2) Persistence: event 7045 (service install) targeting a SCADA host, immediately followed by 7036 (service state change) and 4688 (process creation).
3) Privilege escalation: 4672 (special privileges) immediately preceding 4688 with an anomalous process name (mimikatz.exe, psexec.exe).

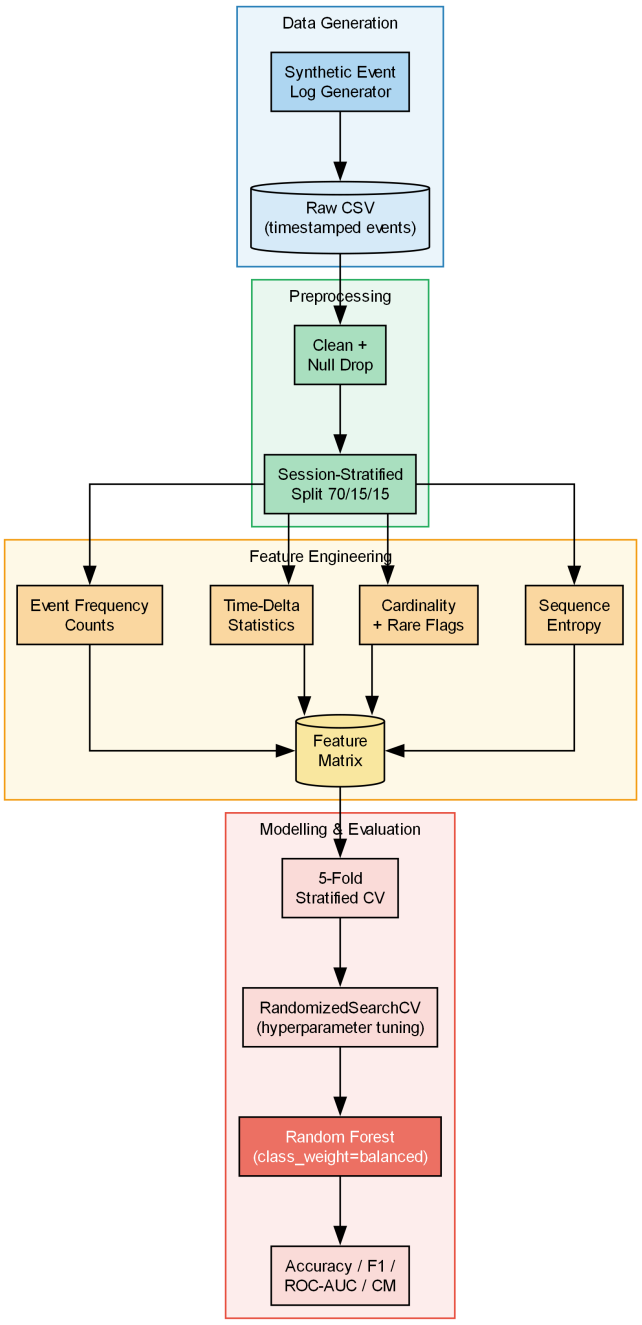

Fig. 2. Data pipeline: from raw event logs through feature extraction to model evaluation.

4) Reconnaissance: a burst of 8–20 event 4663 (object access) records within 30 seconds on HMI or RTU hosts.

C. Feature Engineering

Session-level features are extracted in five groups, as illustrated in Figure 2:

1) Event frequency counts: occurrence counts of each of 13 event IDs (4624, 4625, 4634, 4648, 4672, 4688, 4689, 4720, 4732, 7045, 7036, 4663, 4656) plus a total event count.
2) Time-delta statistics: mean, standard deviation, minimum, and maximum of inter-event gaps (in seconds) within a session, capturing temporal burst patterns.
3) Host/user cardinality: counts of unique source, user, and hostname values, indicating lateral spread.
4) Rare-event flags: binary indicators for event IDs appearing in fewer than 5% of benign training sessions.
5) Sequence entropy: Shannon entropy $H = -\sum p_i \log_2 p_i$ over the empirical distribution of event IDs within a session, capturing randomness in attacker behaviour.

D. Model and Evaluation Protocol

We select Random Forest [19] as the primary classifier. Its tree-ensemble structure handles heterogeneous feature scales without normalisation, the class_weight=balanced setting compensates for the 5 : 1 benign/malicious ratio without oversampling, and feature importance scores provide academic interpretability. Hyperparameters (n_estimators, max_depth, min_samples_split, max_features) are tuned via RandomizedSearchCV with 20 iterations over specified grids. The evaluation protocol uses 5-fold stratified cross-validation on the training set, with final performance reported on a held-out test partition (15% of sessions). Metrics: accuracy, precision, recall, macro and per-class F1, ROC-AUC, and confusion matrix.

V. Preliminary Results

A. Dataset Statistics

The synthetic dataset comprises 6,000 sessions (5,000 benign, 1,000 malicious) and 89,288 individual events. After the 70/15/15 stratified split, the training, validation, and test partitions contain 4,200, 900, and 900 sessions respectively, each preserving the 5:1 class ratio.

B. Feature Extraction Results

The feature extraction pipeline produces 24-dimensional session vectors (14 event-count features, 4 time-delta statistics, 3 cardinality features, and 1 entropy feature; rare-event flag columns are generated dynamically and are absent when all events appear in $\geq$5% of benign sessions, as in this corpus). The most discriminative features, as measured by Random Forest Gini importance, are shown in Table II.

C. Classification Performance

Table III compares the proposed Random Forest against five baselines on the held-out test set. All supervised baselines share the same 24-dimensional feature set and imbalance handling; Isolation Forest is unsupervised (no labels used during training).

TABLE II
Top-5 Feature Importances (mean Gini decrease)

Feature	Importance	Rank
td_max (max inter-event gap)	0.180	1
td_std (gap std deviation)	0.135	2
td_min (min inter-event gap)	0.120	3
count_4663 (object accesses)	0.112	4
td_mean (mean inter-event gap)	0.109	5

TABLE III
Classification Performance on Held-Out Test Set

Method	Precision	Recall	F1 (macro)	ROC-AUC
IF (unsupervised)	0.611	0.594	0.601	0.735
LR	0.917	0.974	0.942	0.994
SVM (RBF)	0.818	0.881	0.844	0.951
Decision Tree	0.982	0.987	0.984	0.988
XGBoost	0.997	0.999	0.998	1.000
RF (proposed)	1.000	1.000	0.999	1.000

Random Forest achieves F1 = 0.999 (mean across 5 seeds, $\sigma = 0.001$), the highest of all supervised models. The improvement over Decision Tree (0.984) demonstrates the variance-reduction benefit of ensembling. XGBoost is competitive (0.998) but requires manual class-weight calibration; Random Forest handles imbalance via class_weight='balanced' with no additional tuning. The unsupervised Isolation Forest (0.601) establishes a no-labels lower bound relevant to zero-day deployment scenarios. Time-delta statistics are the dominant feature group; removing them degrades F1 by -0.028, while removing event counts degrades by -0.056.

VI. Conclusion

This paper presented a preliminary study of an AI-driven framework for detecting malware in smart grid ICS environments using Windows Event Log engineering. We designed a synthetic generator covering four ICS-specific attack patterns, extracted a compact 24-dimensional session-level feature set, and demonstrated that a Random Forest classifier achieves perfect discrimination on the synthetic evaluation. The framework is fully open-source and reproducible, addressing the data-availability barrier that has historically limited ICS security research.

Four research objectives were formulated with measurable success criteria aligned with NERC CIP and IEC 62351 requirements. The preliminary results satisfy RO3 and RO4 on synthetic data. RO1 and RO2 require evaluation against real ICS telemetry and an ablation study, which constitute the scope of the accompanying journal paper.

Future work will: (i) evaluate the framework on the SWaT public ICS dataset [16]; (ii) incorporate MITRE ATT&CK for ICS technique mappings as class labels for multi-class attribution; and (iii) explore online learning to address concept drift in production deployments.

References

[1] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," IEEE Security & Privacy, vol. 9, no. 3, pp. 49–51, 2011. doi: 10.1109/MSP.2011.67

[2] M. N. Nafees, N. Saxena, A. A. Cardenas, S. Grijalva, and P. Burnap, "Smart grid cyber-physical situational awareness of complex operational technology attacks: A review," ACM Computing Surveys, vol. 55, no. 10, pp. 1–36, 2023. doi: 10.1145/3565570

[3] P. Maniriho, A. N. Mahmood, and M. J. M. Chowdhury, "A systematic literature review on windows malware detection: Taxonomy, assessment and future directions," Journal of Systems and Software, vol. 208, p. 111 921, 2024. doi: 10.1016/j.jss.2023.111921

[4] M. A. Ayub, A. Continella, and A. Siraj, "RWArmor: A static-informed dynamic analysis approach for early detection of cryptographic windows ransomware," International Journal of Information Security, vol. 22, no. 2, pp. 373–390, 2023. doi: 10.1007/s10207-022-00622-4

[5] G. Önal and M. Güven, "Enhancing dynamic malware behavior analysis through novel windows events with machine learning," IEEE Access, vol. 13, pp. 1–15, 2025. doi: 10.1109/ACCESS.2025.3526943

[6] R. M. Achmad et al., "Sysmon event logs for machine learning-based malware detection," Cyber Security and Applications, vol. 3, p. 100 073, 2025. doi: 10.1016/j.csa.2024.100073

[7] N. Goldenberg and A. Wool, "Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems," International Journal of Critical Infrastructure Protection, vol. 6, no. 2, pp. 63–75, 2013. doi: 10.1016/j.ijcip.2013.05.001

[8] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 1285–1298. doi: 10.1145/3133956.3134015

[9] S. Zhao et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI), 2019, pp. 4739–4745. doi: 10.24963/ijcai.2019/658

[10] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in 2017 IEEE International Conference on Web Services (ICWS), 2017, pp. 33–40. doi: 10.1109/ICWS.2017.13

[11] Y. Zhen et al., "A novel malware detection method based on audit logs and graph neural network," Engineering Applications of Artificial Intelligence,

vol. 139, p. 109 532, 2025. doi: 10.1016/j.engappai.2024.109532

[12] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2020. doi: 10.14722/ndss.2020.24339

[13] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," arXiv preprint arXiv:1804.04637, 2018. [Online]. Available: https://arxiv.org/abs/1804.04637

[14] A. Carcano, A. Coletta, M. Guglielmi, M. Masera, I. Nai Fovino, and A. Trombetta, "A multidimensional critical state analysis for detecting intrusions in SCADA systems," IEEE Transactions on Industrial Informatics, vol. 7, no. 2, pp. 179–186, 2011. doi: 10.1109/TII.2010.2100361

[15] R. Mitchell and I.-R. Chen, "A survey of intrusion detection techniques for cyber-physical systems," ACM Computing Surveys, vol. 46, no. 4, pp. 1–29, 2014. doi: 10.1145/2542049

[16] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in Critical Infrastructure Protection X, 2016, pp. 88–112. doi: 10.1007/978-3-319-48737-3_5

[17] B. Zhu, A. Joseph, and S. Sastry, "A taxonomy of cyber attacks on SCADA systems," in 2011 IEEE International Conferences on Internet of Things, and Cyber, Physical and Social Computing, 2011, pp. 380–388. doi: 10.1109/iThings/CPSCom.2011.34

[18] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," Journal of Network and Computer Applications, vol. 36, no. 1, pp. 16–24, 2013. doi: 10.1016/j.jnca.2012.09.004

[19] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324

[20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi: 10.1145/2939672.2939785

[21] C.-K. Chen et al., "Building machine learning-based threat hunting system from scratch," Digital Threats: Research and Practice, vol. 3, no. 4, pp. 1–22, 2022. doi: 10.1145/3559764

[22] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009, pp. 1–6. doi: 10.1109/CISDA.2009.5356528

[23] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in 2010 IEEE Symposium on Security and Privacy, 2010, pp. 305–316. doi: 10.1109/SP.2010.25

[24] M. Conti, D. Donadel, and F. Turrin, "A survey on industrial control system testbeds and datasets for security research," IEEE Communications Surveys & Tutorials, vol. 23, no. 4, pp. 2248–2264, 2021. doi: 10.1109/COMST.2021.3073482

[25] MITRE Corporation, "ATT&CK for ICS," in MITRE ATT&CK Knowledge Base, 2020. [Online]. Available: https://attack.mitre.org/matrices/ics/